

Del perceptrón a las convoluciones

Todo lo que traen los cuadernos 1 al 7: fórmulas, operaciones paso a paso, valores reales de cada salida y las trampas que pueden salir en el examen.



Ocho cuadernos, una sola historia: de una neurona a una CNN

C1 • El perceptrón

Neurona, escalón, regla de aprendizaje, sklearn

C1.1 • Perceptrón práctica

Pacientes, MinMaxScaler, accuracy y recall

C2 • TensorFlow y Keras

Tensores, softmax, derivadas, descenso del gradiente

C3 • Regresión en Keras

3 formas de declarar, compile/fit, °C → °F

C4 • Redes densas

Activaciones, pérdidas, batch, overfitting, círculos

C5 • Regresión gasolina

Auto MPG, one-hot, escalado, métricas

C6 • Procesamiento de imágenes

Píxeles, BGR/RGB, kernels, filtros, Otsu

C7 • Redes convolucionales

Conv2D, pooling, fórmulas de tamaño y parámetros

Cuaderno 8 (transferencia de aprendizaje) va después de convoluciones: queda como apéndice al final. Los dos archivos del Cuaderno 1 que subiste son idénticos (mismo contenido byte a byte).

PLAN DE ESTUDIO

Cinco días, un bloque de cuadernos por día

Día	Cuadernos	Qué dominar (practicar a mano)	Dónde
Día 1	C1 + C1.1	Hacer a mano $z = w \cdot x + b$ y la actualización de pesos con los números del cuaderno	Slides 4–12
Día 2	C2 + C3	Tensores y ejes, softmax a mano, derivadas y regla del descenso del gradiente	Slides 13–20
Día 3	C4 + C5	Tabla activación $\leftrightarrow$ pérdida, contar parámetros, leer overfitting en curvas	Slides 21–28
Día 4	C6 + C7	Convolución a mano, fórmula de tamaño de salida y parámetros de Conv2D	Slides 29–38
Día 5	Repaso	Trampas del examen + formulario + cuestionario de práctica	Slides 39–41

Método: lee la diapositiva, tapa los números y recalculalos tú. Si te sale igual que el cuaderno, lo dominas.

Entradas por pesos, se suman con el sesgo y el escalón decide



$$z = w_1x_1 + w_2x_2 + b$$

$$\hat{y} = 1 \text{ si } z > 0, \text{ si no } 0$$

Rosenblatt, 1958. Clasificación binaria. Solo resuelve problemas **linealmente separables** (una recta o hiperplano separa las clases): no puede con XOR. Varias capas de perceptrones = MLP, que sí resuelve problemas no lineales.

¿Éxito en Ingeniería? Nota IA y PGA predicen si el estudiante se gradúa

30 estudiantes con 2 características normalizadas entre 0 y 1:

Característica	Qué es	Media
Nota_IA	Calificación en IA	0.4917
PGA	Promedio General Acumulado	0.5011
estado	1 = se gradúa · 0 = se retira	15 y 15

Etiquetas: **clases** (y). Datos: **alumnos** (X), un array de forma (30, 2).

Graduados arriba a la derecha, retirados abajo a la izquierda: una recta los separa, por eso el perceptrón sí puede resolverlo.

30

registros · len(clases) = len(alumnos) = 30

2

características de entrada → 2 pesos + 1 sesgo

Con pesos al azar el perceptrón clasifica a todos como graduados

Pesos y sesgo aleatorios con `np.random.seed(42)` y `uniform(-1, 1)`:

Parámetro	Valor
w_1	-0.2509
w_2	0.9014
b	0.4640

Predicción sin entrenar, operación completa:

$$\begin{aligned} [0.4, 0.3]: z &= -0.2509 \cdot 0.4 + 0.9014 \cdot 0.3 + 0.4640 \\ z &= -0.1004 + 0.2704 + 0.4640 = 0.634 \rightarrow \hat{y} = 1 \end{aligned}$$

$$\begin{aligned} [0.8, 0.9]: z &= -0.2007 + 0.8113 + 0.4640 = 1.075 \rightarrow \hat{y} \\ &= 1 \end{aligned}$$

El alumno $[0.4, 0.3]$ debería ser 0 (se retira): la predicción es errada porque los pesos no están optimizados. Por eso hay que entrenar.

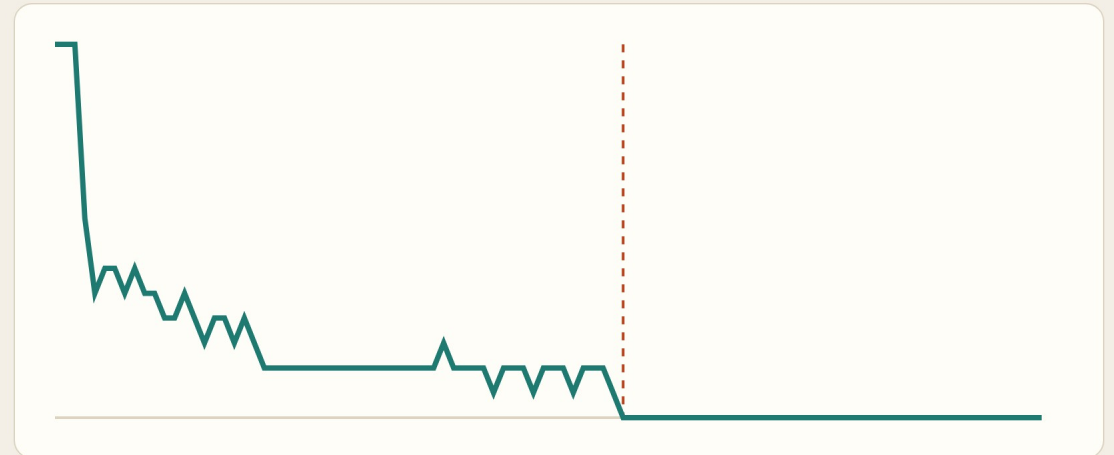
La regla del perceptrón corrige los pesos en proporción al error

$$\begin{aligned}\text{error} &= y - \hat{y} \\ w &\leftarrow w + \eta \cdot \text{error} \cdot x \\ b &\leftarrow b + \eta \cdot \text{error}\end{aligned}$$

Error total por época = $\sum \text{error}^2$ (con salidas 0/1 es el número de fallos).

$\eta = 0.01 \cdot 100$ épocas · recorre las 30 instancias en cada época.

Una **época** = una pasada completa por todos los datos de entrenamiento.



Época 1: 15 errores → época 4: 8 → época 22: 2 → **época 58: 0** (el texto del cuaderno dice «desde la 65», pero la salida muestra 0 desde la 58).

Con los pesos aprendidos el alumno [0.4, 0.3] ya se clasifica como retirado

Parámetro	Entrenado
w_1	0.5004
w_2	0.0851
b	-0.2680

$$\text{frontera: } w_1x_1 + w_2x_2 + b = 0$$

$$x_2 = -(w_1x_1 + b) / w_2$$

Alumno	$z = w \cdot x + b$	$\hat{y}$	Lectura
[0.3, 0.5]	-0.075	0	se retira
[0.8, 0.9]	0.209	1	se gradúa
[0.6, 0.7]	0.092	1	se gradúa
[0.4, 0.3]	-0.042	0	se retira

Ejemplo: $[0.3, 0.5] \rightarrow 0.5004 \cdot 0.3 + 0.0851 \cdot 0.5 - 0.2680 = 0.1501 + 0.0426 - 0.2680 = -0.0753 < 0 \rightarrow 0$.

La gráfica de zonas pinta una malla de 0 a 1 con paso 0.05: azul donde predice 1, rojo donde predice 0.

Tasa de aprendizaje y épocas deciden si el modelo converge

Tasa de aprendizaje η

Controla el tamaño del ajuste. Rango típico 0.001 a 0.1; empezar entre 0.01 y 0.1. Muy alta (≥ 0.5): oscila y puede no converger. Muy baja (≤ 0.0001): aprende muy lento. Si el error no baja, súbela; si oscila, bájala.

Épocas

Pocas $\rightarrow$ subentrenamiento (underfitting). Demasiadas $\rightarrow$ sobreentrenamiento (overfitting). Regla: empezar con 100 o 500 y mirar la curva de error.

Técnicas avanzadas

Validación cruzada (train/validación), decay del learning rate, grid search y random search para buscar hiperparámetros.

Analogía: η = tamaño del paso · épocas = número de pasos dispuestos a dar

Perceptron de sklearn en tres líneas

```
from sklearn.linear_model import Perceptron
p = Perceptron(verbose=True)
p.fit(X_train, y_train)
p.predict([[0.2,0.2],[0.8,0.8]]) → [0., 1.]
```

Parámetro	Default	Significado
max_iter	1000	épocas máximas
eta0	1.0	tasa de aprendizaje inicial
tol	1e-3	mejora mínima para seguir
random_state	None	semilla (reproducibilidad)
early_stopping	False	parar con validación

Configuración	Épocas	coef_	intercept_
default	7	—	-1.0
eta0=0.001	6	[0.00072, 0.001193]	-0.001
max_iter=500, rs=123	7	[0.83, 1.123]	-1.0

Atributos: **coef_** (pesos), **intercept_** (bias), **n_iter_** (épocas), **n_features_in_** = 2, **classes_** = [0., 1.].

¿Por qué 7 épocas si el error ya es 0 en la 2? Se detiene cuando la pérdida no mejora más que tol durante 5 épocas seguidas (**n_iter_no_change=5**): 2 + 5 = 7.

Del CSV al modelo: limpiar, dividir, escalar, entrenar, evaluar

Predecir **problema_cardiaco** con **edad** y **colesterol** (pacientes.csv).

- `df.dropna(inplace=True)` → borra nulos; se guarda `cardiaco_limpio.csv`
- `X = df[['edad','colesterol']]` · `y = df['problema_cardiaco']`
- `train_test_split(test_size=0.2, random_state=42, stratify=y)`
- MinMaxScaler: **fit_transform** en train, solo **transform** en test
- `joblib.dump(scaler, 'scaler.joblib')`
- `Perceptron(tol=1e-3, random_state=42)` → accuracy y recall

MinMax: $x' = (x - \min) / (\max - \min) \rightarrow [0, 1]$

Estándar: $z = (x - \mu) / \sigma \rightarrow \text{media } 0, \text{ desv. } 1$

stratify=y conserva la proporción de clases en train y test.
Ajustar el escalador solo con train evita fuga de datos (data leakage).

La matriz de confusión da todas las métricas

	Predice 1	Predice 0
Real 1	VP	FN
Real 0	FP	VN

$\text{Accuracy} = (\text{VP} + \text{VN}) / \text{total}$

$\text{Precision} = \text{VP} / (\text{VP} + \text{FP})$

$\text{Recall} = \text{VP} / (\text{VP} + \text{FN})$

¿Por qué recall en medicina?

Un falso negativo (enfermo diagnosticado como sano) retrasa el tratamiento y puede costar la vida. Es preferible tener falsos positivos. Recall es robusto al desbalance: un modelo que dice «sano» a todos tiene accuracy alto pero recall 0.

Mínimo aceptable

Condiciones críticas (cáncer, cardiopatía aguda): recall de 90% o más.
No hay número mágico: lo deciden expertos médicos y reguladores.

Keras, TensorFlow, PyTorch y Scikit-learn: qué hace cada uno

Keras

Alto nivel, para prototipar rápido. Corre sobre TensorFlow. Capas y modelos.

TensorFlow

De Google. Bajo nivel: calcula gradientes automáticamente. Mucho control, proyectos grandes.

PyTorch

Grafos computacionales dinámicos, fuerte aceleración en GPU.

Scikit-learn

Clasificación, regresión y agrupamiento. Trabaja con NumPy y SciPy.

Un **tensor** es un arreglo multidimensional con dos atributos:

```
shape → forma en cada eje · dtype →  
float32, int32...
```

tf.constant es inmutable; **tf.Variable** se puede modificar (pesos):

```
var.assign([1,2,3]) → [1,2,3]  
var.assign_add([1,1,1]) → [2,3,4]
```

Ejes, concatenación y reducciones con el tensor x

```
x = tf.constant([[1., 2., 3.],  
[4., 5., 6.]])  
shape (2, 3) · dtype float32
```

Operación	Resultado
$x + x$	[[2,4,6],[8,10,12]]
$5 * x$	[[5,10,15],[20,25,30]]
<code>tf.transpose(x)</code>	forma (3, 2)
$x @ \text{tf.transpose}(x)$	[[14, 32], [32, 77]]
<code>tf.concat([x,x,x], axis=0)</code>	forma (6, 3)
<code>tf.concat([x,x**2,x**3], axis=1)</code>	forma (2, 9)

Reducción	Resultado
<code>reduce_mean(x, axis=1)</code>	[2., 5.] (por fila)
<code>reduce_mean(x, axis=0)</code>	[2.5, 3.5, 4.5]
<code>reduce_sum(x)</code>	21.0

`axis=0` → opera bajando por filas
(un valor por columna)
`axis=1` → opera a lo largo de la fila
(un valor por fila)

Producto matricial: $[1,2,3] \cdot [1,2,3] = 1+4+9 = 14$; $[1,2,3] \cdot [4,5,6] = 4+10+18 = 32$; $[4,5,6] \cdot [4,5,6] = 16+25+36 = 77$.

Convertir números en probabilidades

$$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

Paso	z = [2.0, 1.0, 0.1]
1. Exponenciales	[7.389, 2.718, 1.105]
2. Suma	11.212
3. Dividir	[0.659, 0.242, 0.099]

Las salidas suman 1 → probabilidades. Se usa en la salida de **clasificación multiclase**.

$$\sigma(x) = 1 / (1 + e^{-x})$$

Sigmoide: un solo valor en (0, 1). Salida de **clasificación binaria**. $\sigma(0) = 0.5$ · $\sigma(2) = 0.881$ · $\sigma(-2) = 0.119$.

`tf.nn.softmax(x, axis=1)` aplicado a cada fila de x:

[1, 2, 3] → [0.090, 0.245, 0.665]
[4, 5, 6] → [0.090, 0.245, 0.665]

¡Ambas filas dan lo mismo! Softmax solo depende de las diferencias entre valores.

La derivada dice hacia dónde moverse para bajar el error

$$f(x) = 3x^3 + 2x^2 - 4x - 2$$

$$f'(x) = 9x^2 + 4x - 4$$

$$f''(x) = 18x + 4$$

x	f(x)	f'(x)	f''(x)
2	22	40	40
7.5	1346.125	532.25	139
0.481	-3.127	≈ 0.006	12.658

En $x = 0.481$ la pendiente es casi 0 y $f'' > 0 \rightarrow$ es (casi) un **mínimo local**.
 $f'' > 0$: cóncava hacia arriba; $f'' < 0$: cóncava hacia abajo.

```
x = tf.Variable(7.5)
with tf.GradientTape() as tape:
    y = f(x)
    dy_dx = tape.gradient(y, x)
```

GradientTape graba las operaciones sobre variables y las reproduce hacia atrás con la regla de la cadena. Dos cintas anidadas $\rightarrow$ segunda derivada.

@tf.function convierte la función en grafo: el `print('Tracing')` sale solo la primera vez; con otro dtype o forma se traza de nuevo. **tf.Module** + `tf.saved_model.save` exporta: `MyModule(3).multiply([1,2,3])` $\rightarrow$ `[3,6,9]`; recargado con `[4,5,6]` $\rightarrow$ `[12,15,18]`.

Un modelo cuadrático entrenado a mano y con Keras

Datos sintéticos: 201 puntos de -2 a 2 con ruido normal (media 0, varianza 1):

```
y = x2 + 2x - 5 + ruido
X = [x, x2] → shape (201, 2)
```

```
 $\hat{y} = w_1x + w_2x^2 + b$ 
MSE =  $(1/n) \sum (\hat{y}_i - y_i)^2$ 
 $\partial \text{MSE} / \partial w_1 = (2/n) \sum (\hat{y}_i - y_i) \cdot x_{1i}$ 
 $w \leftarrow w - \eta \cdot \partial \text{MSE} / \partial w$ 
```

Valores ideales que debe aprender: $w_1 \approx 2$, $w_2 \approx 1$, $b \approx -5$.

A mano con tf.Module

Neurona: W (2×1) aleatoria normal con seed 42, $b = 0$. 200 épocas, $\eta = 0.05$. $W.\text{assign_sub}(\eta \cdot \text{grad})$. La gráfica MSE vs w_1 ($b = -5$, $w_2 = 1$ fijos) es una parábola; la tangente en $w_1 = 0$ muestra la pendiente.

Con Keras

$\text{Lambda}(\text{stack}[x, x^2]) + \text{Dense}(1) \rightarrow 3$ parámetros.
`compile(loss=MSE, optimizer=SGD(learning_rate=0.01));`
`fit(epochs=100, batch_size=32) \rightarrow 7` lotes por época ($201/32 = 6.3$). Se guarda con `.save('modelo_keras.keras')`.

Tres formas de declarar la misma red y cómo contar sus parámetros

```
# A · lista
Sequential([Input((20,)),
Dense(64,'relu'), Dense(32,'relu'),
Dense(1,'sigmoid')])
```

```
# B · .add()
m = Sequential(); m.add(Dense(64,...))
```

```
# C · API funcional
x = Dense(64)(entradas)
Model(inputs=entradas, outputs=salida)
```

parámetros Dense = entradas × neuronas
+ neuronas

Capa	Cálculo	Parámetros
Dense 64	$20 \cdot 64 + 64$	1 344
Dropout 0.2	sin pesos	0
Dense 32	$64 \cdot 32 + 32$	2 080
Dense 1	$32 \cdot 1 + 1$	33
Total		3 457

La API funcional es la más flexible (varias entradas o salidas, ramas: ResNet, Inception). Dropout(0.2) apaga el 20% de neuronas al entrenar para evitar sobreajuste.

Compile define cómo aprende; fit ejecuta el aprendizaje

compile()

optimizer: Adam(learning_rate=0.001).
Alto → inestable o diverge; bajo → lento o mínimo local.

loss: binaria BinaryCrossentropy;
multiclase CategoricalCrossentropy
(one-hot) o
SparseCategoricalCrossentropy
(enteros); regresión MSE o MAE.

metrics: accuracy, precision, recall.

fit()

Forward pass → pérdida → backward
pass → actualizar pesos.

epochs: pasadas completas.

batch_size: muestras antes de
actualizar. **validation_data:** (X_val,
y_val). **verbose:** 0 silencioso, 1 barra, 2
una línea por época.

Después

history.history guarda loss, val_loss,
accuracy, val_accuracy por época.
evaluate(X_val, y_val) → pérdida
y métrica.
predict() → probabilidades; clase =
(prob > 0.5).

Ejemplo teórico: 1000 muestras × 20 características, y = (suma de X > 0), 800 train / 200 validación, 30 épocas, batch 32.

Una sola neurona aprende a convertir Celsius a Fahrenheit

$$^{\circ}\text{F} = ^{\circ}\text{C} \times 1.8 + 32$$

12 datos: $^{\circ}\text{C} = [-35, -30, -20, -10, -5, 0, 10, 15, 20, 25, 35, 40] \rightarrow ^{\circ}\text{F} = [-31, -22, -4, 14, 23, 32, 50, 59, 68, 77, 95, 104]$. No se escalan.

```
Sequential([Input(shape=(1,)), Dense(1)])  
Adam(learning_rate=0.01) · loss='mean_squared_error'  
fit(celsius, fahrenheit, epochs=1000)
```

La red debe aprender **peso ≈ 1.8** y **sesgo ≈ 32** . Predicción de $16^{\circ}\text{C} \rightarrow 16 \cdot 1.8 + 32 = 60.8^{\circ}\text{F}$.

2

parámetros: 1 peso + 1 sesgo

7347.6

pérdida (MSE) en la época 1

0.0018

pérdida en la época 1000 (≈ 400 épocas bastan)

Las activaciones aportan la no linealidad

Función	Fórmula	Rango	Uso recomendado
ReLU	$\max(0, x)$	$[0, \infty)$	Capas ocultas (por defecto). Reduce vanishing gradient
Sigmoid	$1 / (1 + e^{-x})$	$(0, 1)$	Salida de clasificación binaria
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	$(-1, 1)$	Ocultas con datos centrados en 0; RNN clásicas
Linear	x	$(-\infty, \infty)$	Salida de regresión (derivada = 1)
Softmax	$e^{z_i} / \sum e^{z_j}$	$(0, 1)$, suman 1	Salida multiclase exclusiva

OJ En Keras, Dense(64) sin activation usa **linear**. Si todas las capas ocultas son lineales, $W_3(W_2(W_1x))$ = una sola transformación lineal: la red no aprende relaciones no lineales.

La pérdida debe coincidir con el problema y con la salida

Problema	Salida	Pérdida	En Keras (loss=)
Regresión	Linear	MSE / MAE	'mse' · 'mae'
Binaria	1 neurona Sigmoid	Binary Crossentropy	'binary_crossentropy'
Multiclase one-hot	N neuronas Softmax	Categorical CE	'categorical_crossentropy'
Multiclase enteros	N neuronas Softmax	Sparse Categorical CE	'sparse_categorical_crossentropy'

$$\text{MSE} = (1/n) \sum (y - \hat{y})^2$$

$$\text{MAE} = (1/n) \sum |y - \hat{y}|$$

$$\text{BCE} = -[y \cdot \log p + (1-y) \cdot \log(1-p)]$$

$$\text{CCE} = -\sum y_i \cdot \log p_i$$

Real 100, predicción 90: error 10 → MSE 100, MAE 10. MSE castiga más los errores grandes y es sensible a outliers.

BCE con y = 1: p = 0.9 → 0.105; p = 0.1 → 2.303. Castiga mucho la confianza equivocada.

Pérdida: se usa al entrenar, es diferenciable, se minimiza con backpropagation. **Métrica:** evalúa (accuracy, precision, recall, AUC) y no entra al gradiente.

Cuántos datos por actualización y cómo dividir el dataset

$$\text{Lotes por época} = \lceil n / \text{batch_size} \rceil$$

Datos	Batch	Lotes
10 000	32	≈ 312 (313 exacto)
500 círculos	32	15.625 → 16
400 (con val 20%)	32	12.5 → 13
50 000 CIFAR-10	32	1563

Batch completo = Batch GD; una muestra = SGD; intermedio = **mini-batch**. Tamaños comunes 16, 32, 64, 128. Muy grande generaliza peor; muy pequeño es inestable.

Conjunto	¿Aprende?	¿Ajusta hiperparámetros?	¿Evalúa final?
Train	Sí	No	No
Validation	No (directo)	Sí	No
Test	No	No	Sí

Tamaño dataset	Train	Val	Test
Grande	70%	15%	15%
Mediano	70%	20%	10%
Pequeño	80%	10%	10%

Dataset pequeño: K-Fold (entrena K veces cambiando la validación). En series de tiempo no mezclar futuro con pasado. Capas ocultas: potencias de 2, embudo 128 → 64 → 32.

Detectar el overfitting y frenarlo a tiempo

Época	Loss train	Loss val
5	0.40	0.42
10	0.30	0.35
15	0.20	0.50 ↑

Overfitting: `loss_train` ↓ y `val_loss` ↑

Otra señal: accuracy train 98% vs validación 82%. Causas: modelo muy complejo, pocos datos, demasiadas épocas, sin regularización.

```
EarlyStopping(monitor='val_loss',  
patience=5, min_delta=0.001,  
restore_best_weights=True, mode='min')
```

Parámetro	Qué hace
monitor	métrica a vigilar (<code>val_loss</code> recomendado)
patience	épocas sin mejora antes de parar (3–10)
min_delta	mejora mínima que cuenta como mejora
restore_best_weights	vuelve a los pesos de la mejor época
mode	'min' para pérdida, 'max' para accuracy

Otros remedios: L2 (`kernel_regularizer=l2(0.01)`), Dropout(0.5), menos capas o neuronas, más datos o data augmentation.

Una sola neurona no puede separar círculos concéntricos

```
make_circles(n_samples=500, factor=0.5,  
noise=0.05) # seed 42
```

500 puntos, 2 características (X1, X2), $\text{df.shape} = (500, 3)$. Círculo interior = clase 1, exterior = clase 0. **No es linealmente separable.**

```
Sequential([Input(shape=(2,)),  
Dense(1, activation='sigmoid')])  
adam · binary_crossentropy · 100 épocas · batch 32
```

Parámetros: $2 \cdot 1 + 1 = 3$. Lotes: $500/32 = 15.625 \rightarrow 16$ (el comentario del cuaderno dice $\text{int}(500/32) = 16$, pero int da 15; lo correcto es redondear hacia arriba).

0.498accuracy final ($\approx$ adivinar al azar) · loss 0.6975

Punto	Prob.	Clase
[-0.7, -0.5]	0.432	0
[-0.9, 0.5]	0.489	0
[-0.5, 0.1]	0.482	0
[0.4, 0.4]	0.546	1
[0.7, 0.7]	0.580	1

Con dos capas ocultas la red separa los círculos al 100%

Capa	Cálculo	Parámetros
Dense 8 relu	$2 \cdot 8 + 8$	24
Dense 4 relu	$8 \cdot 4 + 4$	36
Dense 1 sigmoid	$4 \cdot 1 + 1$	5
Total		65

Modelo	Resultado
2→8→4→1, 500 épocas	acc 1.0 · loss 0.00508
+ EarlyStopping, val 20%	paró en época 166 · 13 lotes
3 entradas + $(X1 \cdot X2)^2$	73 parámetros · acc 1.0 · loss 0.00068

Punto (profundo)	Prob.	Clase
$[-1.0, 0.0]$	0.00001	0
$[-0.5, -0.4]$	0.988	1
$[-0.3, 3]$	0.0	0
$[0, -0.5]$	0.990	1
$[0.8, 0.4]$	0.0017	0

Las capas ocultas con ReLU transforman el espacio para que el círculo se vuelva separable. Agregar la característica $(X1 \cdot X2)^2$ es el truco de kernel polinomial (como en SVM).

- o `model_deep2.layers[1]` es la **segunda** capa oculta (8×4),
- J aunque el código la llama «first hidden layer».
- o

Preparar datos sin fuga: dividir primero, ajustar solo con train

Dataset Auto MPG (Kaggle): predecir **mpg** (millas por galón)
→ regresión.

- 398 filas; '?' → NaN; drop_duplicates
- horsepower a numérico; nulos con la media = **104.47**
- Se elimina 'car name'
- 7 entradas: cylinders, displacement, horsepower, weight, acceleration, model year, origin
- origin se trata como texto (categórica)

Conjunto	Filas
Test (20% de 398)	80
Validación (20% de 318)	64
Train	254

```
6 numéricas → StandardScaler  
origin (3 categorías) → OneHotEncoder  
6 + 3 = 9 columnas → (254, 9)
```

Mediana (numéricas) y moda (categóricas) y el escalador se ajustan solo con train: media ≈ 0 y desviación = 1 en train.

La red se equivoca en promedio 1.76 mpg en datos nuevos

Capa	Salida	Parámetros
Dense relu	128	$9 \cdot 128 + 128 = 1\,280$
Dropout 0.20	128	0
Dense relu	64	$128 \cdot 64 + 64 = 8\,256$
Dropout 0.10	64	0
Dense relu	32	$64 \cdot 32 + 32 = 2\,080$
Dense (lineal)	1	$32 + 1 = 33$
Total		11 649

Adam(0.001) · loss 'mse' · métricas MAE y RMSE · EarlyStopping(patience=15)
→ paró en la época 93.

5.13

MSE en test

1.76

MAE en test (mpg)

2.265

$RMSE = \sqrt{MSE} = \sqrt{5.13}$

Real	Predicho
33.0	33.23
28.0	31.51
19.0	20.42

Una imagen es una matriz de números llamados píxeles

Escala de grises

2D (alto, ancho). Un valor por píxel, 0 a 255 en 8 bits. Ej.: happyface 20×20.

Binaria

Solo dos valores (0 y 255, o 0 y 1): blanco y negro puro.

Color

3D (alto, ancho, canales). logo.jpg → (225, 225, 3); Fortnite → (1080, 1920, 3).

```
a.shape (2, 4, 3) · a.ndim 3 · a.size 24 ·  
a.dtype int64
```

```
concatenate((a, a2), axis=0) → (4, 4, 3)
```

flatten() devuelve una copia plana y **ravel()** una vista (ahorra memoria): de (4, 4, 3) salen 48 valores. Las capas densas solo reciben arreglos planos. dtype uint8 = enteros 0–255; imshow con float espera 0–1.

El orden de los canales cambia el resultado

- OpenCV lee en **BGR**, no RGB. Convertir: `cv.cvtColor(img,`
- J** `cv.COLOR_BGR2RGB)`. Primer píxel del logo: BGR (79, 192, 249) → RGB
- (249, 192, 79).

$$\text{Gris} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

El ojo es más sensible al verde, luego al rojo y menos al azul. El cuaderno 6 también usa 0.2989 / 0.5870 / 0.1140.

$$\begin{aligned} &0.299 \cdot 249 + 0.587 \cdot 192 + 0.114 \cdot 79 \\ &= 74.45 + 112.70 + 9.01 = 196 \end{aligned}$$

196

gris correcto con COLOR_BGR2GRAY sobre la imagen BGR

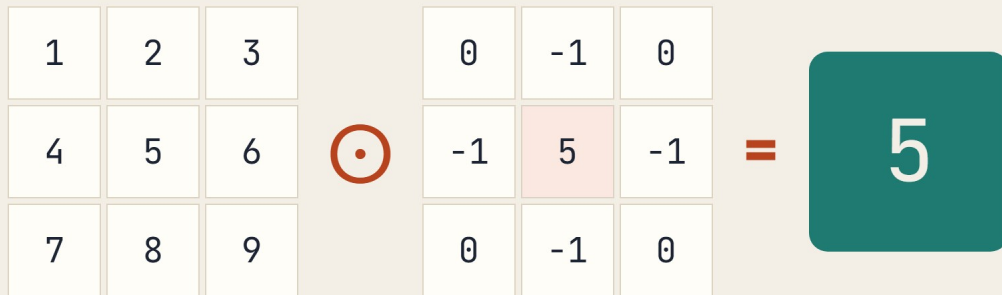
165

si se pasa la imagen RGB a BGR2GRAY: se cruzan R y B

Umbral: `cv.threshold(gris, 150, 255, THRESH_BINARY)` → > 150 = 255, resto 0. `resize` a (50, 50) → `shape` (50, 50, 3).

Convolucionar es multiplicar por el kernel y sumar

Se pone el kernel sobre una vecindad, se multiplica elemento a elemento y se **suma**: ese es el nuevo píxel central.



Nitidez: $-2 - 4 + 5 \cdot 5 - 6 - 8 = 5$

Kernel 3×3	Valores	Resultado sobre I
Promedio (blur)	todos 1/9	$45 / 9 = 5$
Sobel horizontal	$[-1 \ 0 \ 1; -2 \ 0 \ 2; -1 \ 0 \ 1]$	8
Nitidez	$[0 \ -1 \ 0; -1 \ 5 \ -1; 0 \ -1 \ 0]$	5

Sobel: $(-1 \cdot 1 + 1 \cdot 3) + (-2 \cdot 4 + 2 \cdot 6) + (-1 \cdot 7 + 1 \cdot 9) = 2 + 4 + 2 = 8$.

- El cuaderno dice «kernel 5×5 de unos / 25» pero el código
- divide entre **15**: suma $25/15 \approx 1.67 \rightarrow$ la imagen se aclara
- y satura en 255.

Desenfocar, quitar ruido y detectar bordes con distintas ventanas

Media (uniforme)

Mismo peso a todos los vecinos: promedio. `uniform_filter(size=5)` o ventana `np.ones((n,n)) / n²`.

Gaussiano

Más peso al centro, según la normal: $\exp(-(x^2+y^2) / 2\sigma^2)$. Desenfoque más suave.

Mediana

Reemplaza por la mediana de la vecindad. Elimina ruido y **preserva bordes**.

Sobel: $\text{magnitud} = \sqrt{S_x^2 + S_y^2}$

`np.hypot(sobel_x, sobel_y)` hace lo mismo. Resalta cambios bruscos de intensidad: los bordes.

`convolve2d(canal, ventana,
mode='same', boundary='symm')`

Se convoluciona cada canal R, G, B por separado y se apilan (`np.stack axis=2`). Ventanas probadas: 9, 17, 33, 65.

Separar la imagen en regiones: umbral, Otsu y K-Means

Umbral simple

píxel > umbral → 255, si no → 0. Se probaron 100, 120, 128, 138 y 150 sobre la imagen en gris.

Otsu

Prueba los 256 umbrales y elige el que **maximiza** la varianza entre clases $w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$, que equivale a minimizar la varianza dentro de cada clase. Mejor aplicado por canal R, G, B.

K-Means

Agrupar los píxeles en el espacio de color (3 clústeres). Cada píxel toma el color de su centroide. Luego `measure.find_contours` y `approximate_polygon` vectorizan un clúster.

```
im.reshape((h*w, 3)) → KMeans(n_clusters=3).fit → cluster_centers_ → labels_
```

Una CNN aprende sus propios kernels

Conv2D

filtros que aprenden
bordes y texturas

ReLU

no linealidad

MaxPooling

reduce resolución,
toma el máximo

Flatten

3D → vector

Dense

clasifica

Salida

softmax o sigmoide

CIFAR-10: 60 000 imágenes a color de 32×32 en 10 clases.

```
train (50000, 32, 32, 3) · test (10000, 32, 32, 3)  
etiquetas (50000, 1) · normalizar: /255.0 → [0, 1]
```

Clases (índice 0 a 9):

avión, automóvil, pájaro, gato, ciervo, perro, rana, caballo,
barco, camión

Diferencia con la red densa: la CNN detecta características **locales** y aprende representaciones jerárquicas, en vez de conectar cada píxel con cada neurona.

Dos fórmulas explican toda la tabla de model.summary()

$$\text{salida} = (N - K + 2P) / S + 1$$

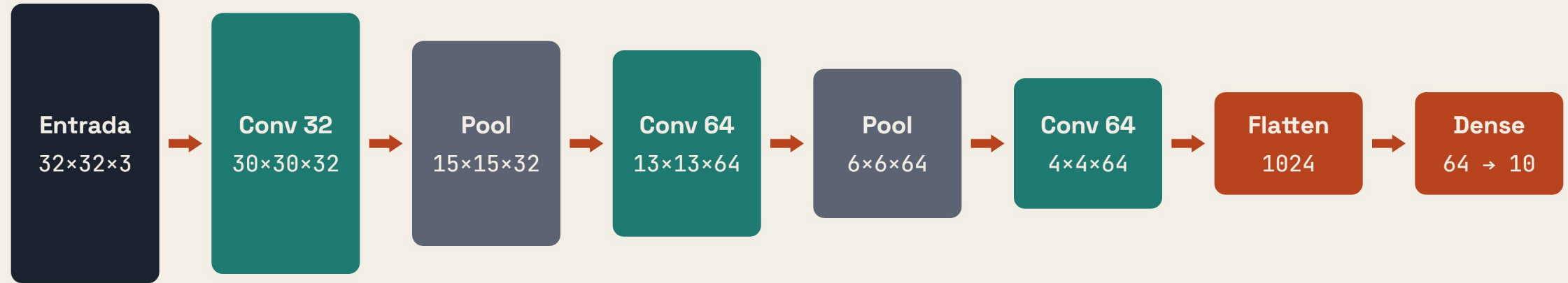
N = tamaño de entrada, K = kernel, P = padding, S = stride.
Keras usa padding 'valid' (P = 0) y S = 1 por defecto.

$$\text{parámetros Conv2D} = (K \cdot K \cdot C_{\text{entrada}} + 1) \cdot \text{filtros}$$

MaxPooling 2×2: divide alto y ancho entre 2 y redondea hacia abajo. No tiene parámetros.

Capa	Forma de salida	Cálculo	Parám.
Conv2D 32, 3×3	(30, 30, 32)	$(3 \cdot 3 \cdot 3 + 1) \cdot 32$	896
MaxPool 2×2	(15, 15, 32)	30/2	0
Conv2D 64, 3×3	(13, 13, 64)	$(3 \cdot 3 \cdot 32 + 1) \cdot 64$	18 496
MaxPool 2×2	(6, 6, 64)	[13/2]	0
Conv2D 64, 3×3	(4, 4, 64)	$(3 \cdot 3 \cdot 64 + 1) \cdot 64$	36 928
Flatten	(1024)	$4 \cdot 4 \cdot 64$	0
Dense 64	(64)	$1024 \cdot 64 + 64$	65 600
Dense 10	(10)	$64 \cdot 10 + 10$	650
Total			122 570

La imagen se encoge en tamaño y crece en profundidad



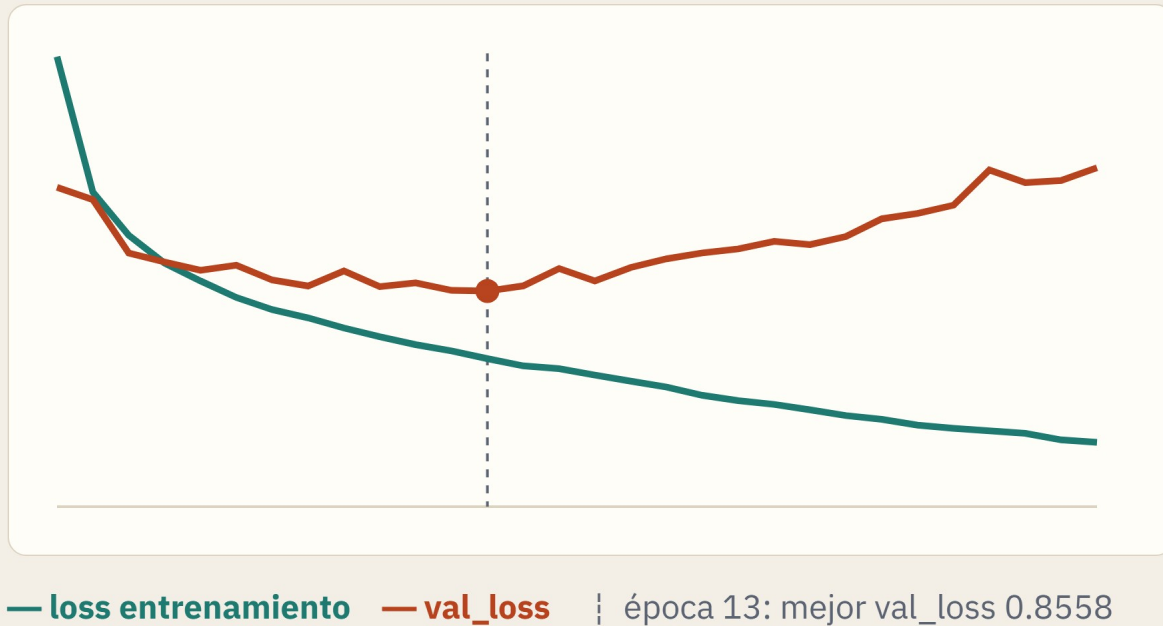
Qué cambia al avanzar

Alto y ancho bajan (32 → 30 → 15 → 13 → 6 → 4) y los canales suben (3 → 32 → 64). La red pasa de ver bordes a ver partes y objetos.

Salida sin activación

Dense(10) entrega **logits**. Por eso la pérdida es `SparseCategoricalCrossentropy(from_logits=True)`: aplica softmax internamente. Etiquetas enteras 0–9 → «Sparse».

Después de la época 13 la CNN memoriza en vez de generalizar



Época	acc	val_acc	val_loss
1	0.341	0.539	1.268
13	0.791	0.710	0.856
30	0.909	0.697	1.346

adam · 30 épocas (el texto dice 10) · 1563 lotes por época.
Validación = test (10 000 imágenes).

Accuracy de test final: 0.697. La brecha 0.909 vs 0.697 y la val_loss que sube desde la época 13 son overfitting: un EarlyStopping habría parado ahí.

Predecir una imagen nueva y guardar el modelo

```
img.resize((32, 32)) → img_to_array → / 255.0
np.expand_dims(axis=0) → (1, 32, 32, 3)
```

La salida son 10 logits. El mayor es 14.29 en el índice 0; $\text{argmax} = 0 \rightarrow$ **avión**. Con softmax esa clase tiene probabilidad ≈ 0.99997 .

Índice	Clase	Logit
0	avión	14.29
8	barco	3.64
2	pájaro	0.98
7	caballo	-11.71

Formato	Cómo
.keras (nativo, recomendado)	model.save('imagenesCNN1.keras')
.h5 (HDF5, legacy)	model.save('imagenesCNN1.h5')
Arquitectura JSON	model.to_json()
Solo pesos	save_weights('...weights.h5')
TensorFlow Lite	TFLiteConverter.from_keras_model

Cargar: `load_model('imagenesCNN1.keras')`. TFLite: entrada (None, 32, 32, 3) $\rightarrow$ salida (None, 10).

- 0 El cuaderno escribe el JSON en 'imagenesCNN1.h5' y
- J sobrescribe el modelo .h5; debería llamarse .json.
- 0

Después de convoluciones: transferencia de aprendizaje

Transferencia básica

Se congela el modelo preentrenado (`trainable = False`) y solo se entrena un clasificador nuevo encima. Pocos datos o datos diferentes; bajo riesgo de overfitting.

Fine-tuning

Se descongelan las capas superiores y se entrenan junto al clasificador con una tasa baja. Datos parecidos a los del preentrenamiento.

Dato (VGG16, perros y gatos)	Valor
Capas	13 conv + 3 densas · ~138 M parámetros
Entrada	224×224×3 · include_top=False → (32, 7, 7, 512)
Base congelada	14 714 688 parámetros
Cabeza nueva	$512 \cdot 128 + 128 = 65\,664 \cdot 128 + 1 = 129$
Fine-tune (fine_tune_at=15 de 19)	7 145 217 entrenables
Datos	2000 train · 1000 validación

Diez detalles de los cuadernos que suelen caer en el examen

C El texto dice que el error llega a 0 en la época 65; la salida
1 muestra 0 desde la 58.

C Softmax de [1,2,3] y de [4,5,6] da lo mismo: [0.090,
2 0.245, 0.665].

C Dense sin activation = linear. Capas lineales apiladas = una sola
4 capa lineal.

C OpenCV lee BGR. BGR2GRAY sobre una imagen RGB da 165 en
6 vez de 196.

C Dense(10) sin softmax → from_logits=True. El texto dice 10
7 épocas; el código usa 30.

C1 sklearn para en la época 7 (no en la 2) por n_iter_no_change = 5.

C4 $\text{int}(500/32) = 15$, no 16. Los lotes se redondean hacia arriba: 16.

C4 layers[1] es la segunda capa oculta, no la primera.

C Kernel de unos dividido entre 15 (no 25): aclara y satura
6 la imagen.

C val_loss mínima en la época 13 y luego sube: overfitting aunque
7 train llegue a 0.91.

Las fórmulas del parcial en una sola hoja

Perceptrón

$$z = \sum w_i x_i + b \cdot \hat{y} = 1 \text{ si } z > 0$$

Regla

$$w += \eta(y - \hat{y})x \cdot b += \eta(y - \hat{y})$$

Gradiente

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

Sigmoide

$$\sigma(x) = 1 / (1 + e^{-x})$$

Softmax

$$e^{z_i} / \sum e^{z_j}$$

ReLU

$$\max(0, x)$$

MSE · RMSE

$$(1/n) \sum (y - \hat{y})^2 \cdot \sqrt{\text{MSE}}$$

BCE

$$-[y \log p + (1-y) \log(1-p)]$$

Dense

$$\text{entradas} \cdot \text{neuronas} + \text{neuronas}$$

Conv2D

$$(K \cdot K \cdot C_{\text{in}} + 1) \cdot \text{filtros}$$

Salida conv

$$(N - K + 2P) / S + 1$$

Gris

$$0.299R + 0.587G + 0.114B$$

MinMax

$$(x - \min) / (\max - \min)$$

Recall

$$VP / (VP + FN)$$